Parallel Computation/Program Issues

Conditions of Parallelism

- **Control Dependence:**
  - **Order of execution cannot be determined before runtime due to conditional statements.**
- **Resource Dependence:**
  - **Concerned with conflicts in using shared resources among parallel tasks, including:**
    - **Functional units (integer, floating point), memory areas, communication links etc.**
- **Bernstein's Conditions of Parallelism:**

Two processes P_1 , P_2 with input sets I_1, I_2 and output sets O_1, O_2 can execute in parallel (denoted by $P_1 \parallel P_2$) if:

$$I_1 \cap O_2 = \varnothing$$
$$I_2 \cap O_1 = \varnothing$$
$$O_1 \cap O_2 = \varnothing$$

i.e no output dependence

Bernstein's Conditions: An Example

- For the following statements P_1, P_2, P_3, P_4, P_5 :
  - Each instruction requires one step to execute
  - Two adders are available

```
P1
Co-Begin
    P1, P3, P5
Co-End
P4
```

$P_1 : C = D \times E$

$P_2 : M = G + C$

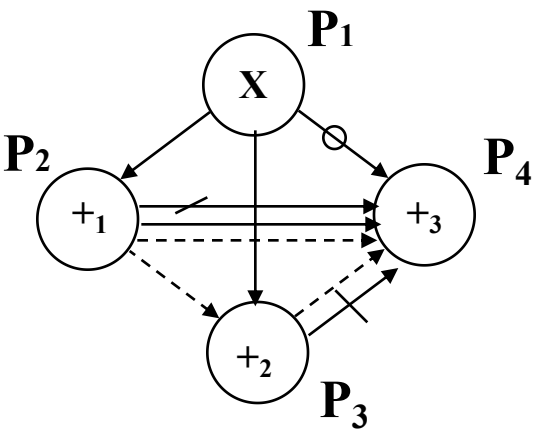
$P_3 : A = B + C$

$P_4 : C = L + M$

$P_5 : F = G \div E$

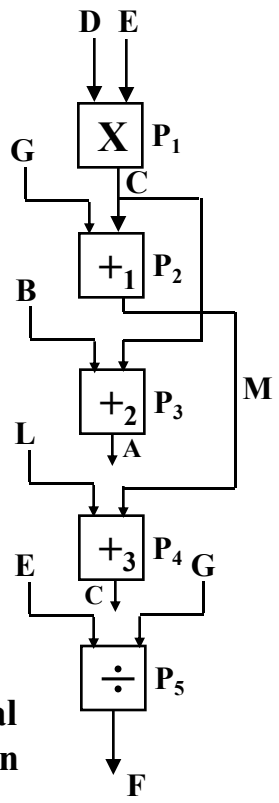
Using Bernstein's Conditions after checking statement pairs:

$P_1 \parallel P_5$, $P_2 \parallel P_3$, $P_2 \parallel P_5$, $P_3 \parallel P_5$, $P_4 \parallel P_5$



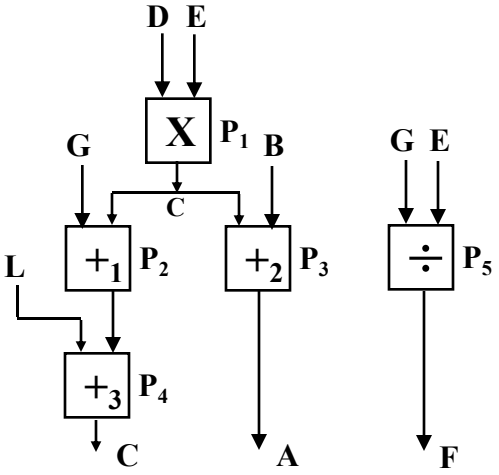
Data dependence (solid lines)
Resource dependence (dashed lines)





Sequential execution

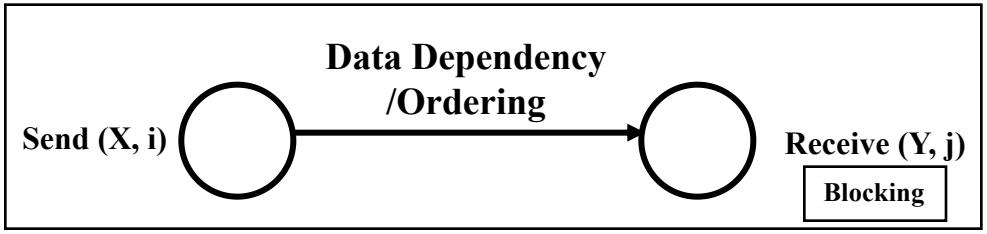
Time



Parallel execution in three steps assuming two adders are available per step

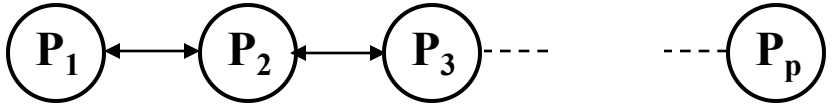
Network Model of Message-Passing Multicomputers

- A network of processors can viewed as a graph G (N, E)
  - Each node $i \in N$ represents a processor
  - Each edge $(i, j) \in E$ represents a two-way communication link between processors i and j.
  - Each processor have its own local memory. No shared memory is available.
  - Operation is synchronous or asynchronous (using message passing).
  - <u>Basic message-passing communication primitives:</u>
    - send(X, i) a copy of data X is sent to processor P_i, execution continues.
    - receive(Y, j) execution of recipient processor is suspended (blocked or waits) until the data from processor P_j is received and stored in Y then execution resumes.



Network Model of Multicomputers

- Routing is concerned with delivering each message from source to destination over the network.
- Additional important network topology parameters:
  - The network diameter is the maximum distance between any pair of nodes (in links or hops).
  - The maximum degree of any node in G
    - Directly connected to how many other nodes
- Example:

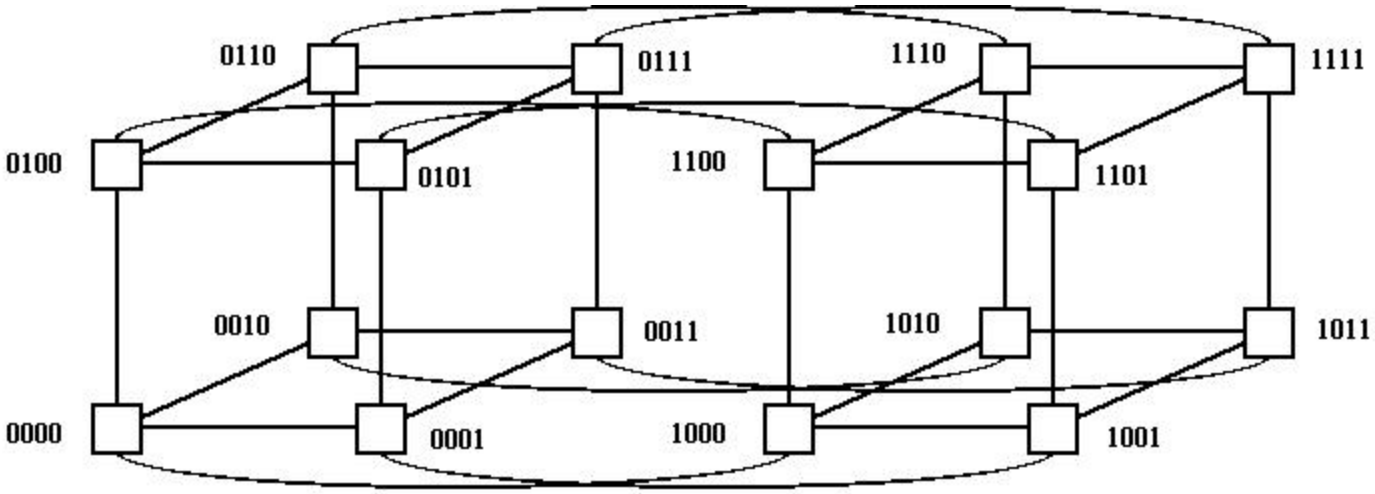


  - Linear array: P processors P_1, …, P_p are connected in a linear array where:
    - Processor P_i is connected to P_{i-1} and P_{i+1} if they exist.
    - Diameter is p-1; maximum degree is 2 (1 or 2).
  - A ring is a linear array of processors where processors P_1 and P_p are directly connected. Degree = 2, Diameter = p/2

Example Network Topology:

A Four-Dimensional (4D) Hypercube

- In a d-dimensional binary hypercube, each of the $N = 2^d$ nodes is assigned a d-bit address.
- Two processors are connected if their binary addresses differ in one bit position.
- Degree = Diameter = $d = \log_2 N$ *here d = 4*



Here: d = Degree = Diameter = 4

Number of nodes = $N = 2^d = 2^4 = 16$ nodes

Dimension order routing example node 0000 to 1111:

$0000 \xrightarrow{1} 0001 \xrightarrow{2} 0011 \xrightarrow{3} 0111 \xrightarrow{4} 1111$ (4 hops)

Creating a Parallel Program

One must: Computational Problem ⟶ Parallel Algorithm ⟶ Parallel Program

- Identify work that can be done in parallel (dependency analysis)
- Partition work and perhaps data among processes (Tasks)
- Manage data access, communication and synchronization

Main goal: Maximize Speedup

$$Speedup\ (p) = \frac{Performance(p)}{Performance(1)}$$

By:
1- Minimizing parallelization overheads
2- Balancing workload on processors

For a fixed size problem:

$$Speedup\ (p) = \frac{Time(1)}{Time(p)}$$

The processor with max. execution time determines parallel execution time

Time (p) **= *Max* (Work + Synch Wait Time + Comm Cost + Extra Work)**

Hardware Vs. Software Parallelism

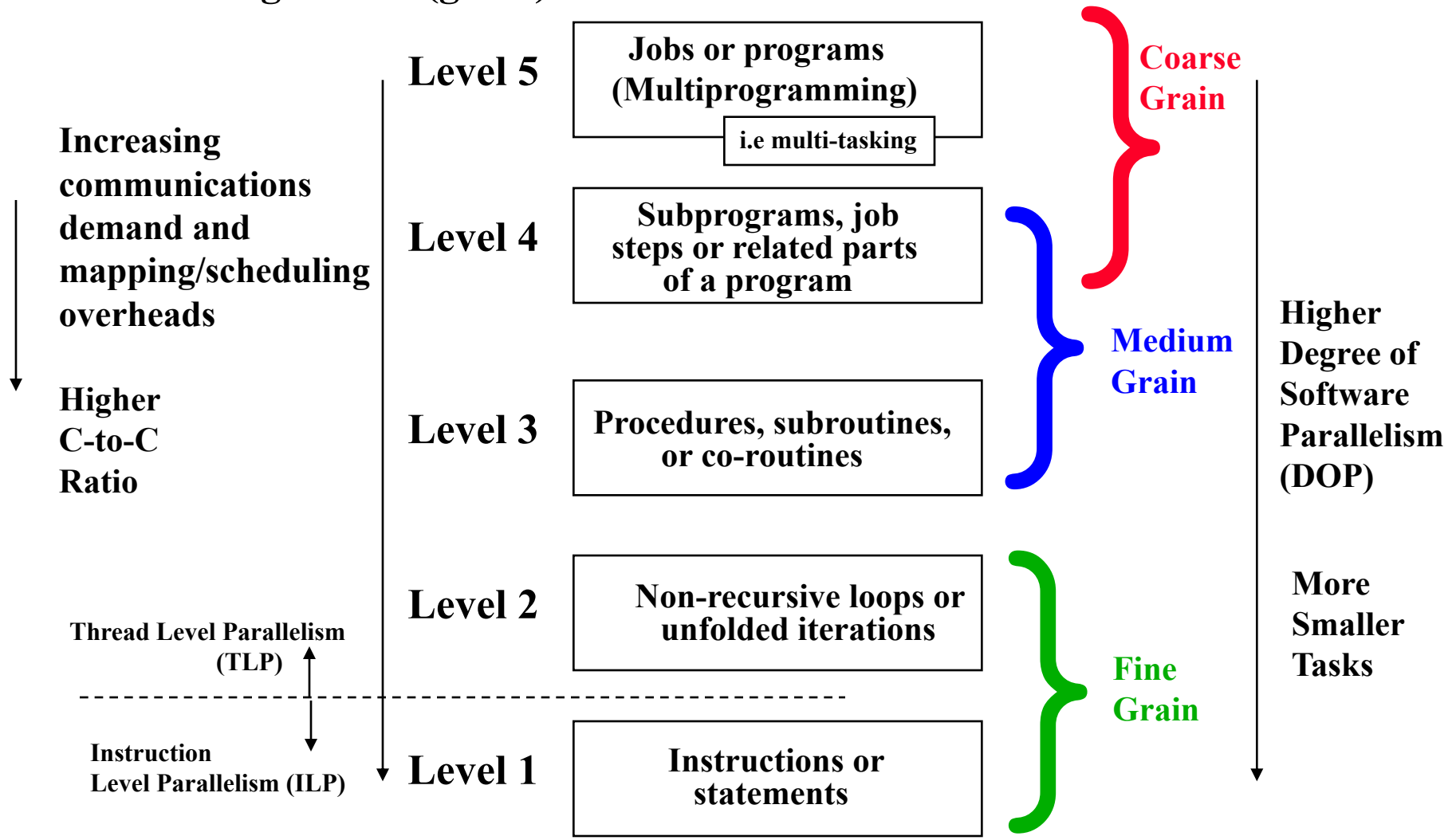
- **Hardware parallelism:** e.g Number of processors
  - Defined by machine architecture, hardware multiplicity (number of processors available) and connectivity.
  - Often a function of cost/performance tradeoffs.
  - Characterized in a single processor by the number of instructions *k* issued in a single cycle (*k*-issue processor).
  - A multiprocessor system with *n* *k*-issue processor can handle a maximum limit of *nk* parallel instructions (at ILP level) or n parallel threads at thread-level parallelism (TLP) level.

- **Software parallelism:**
  - Defined by the control and data dependence of programs.
  - Revealed in program profiling or program dependency (data flow) graph. → i.e. number of parallel tasks at a given time
  - A function of algorithm, parallel task grain size, programming style and compiler optimization.

DOP = Degree of Parallelism

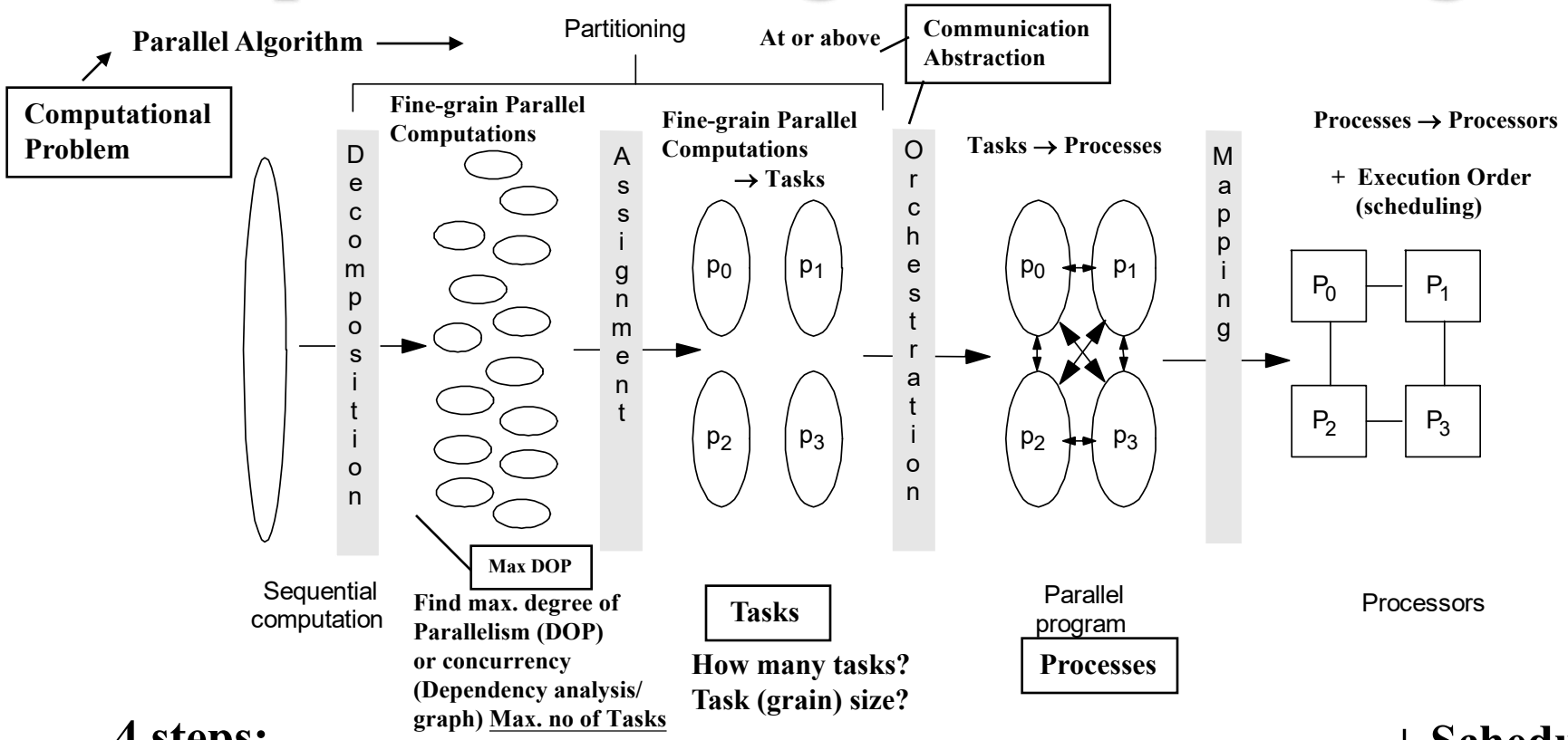
Levels of Software Parallelism in Program Execution

According to task (grain) size



Task size affects Communication-to-Computation ratio (C-to-C ratio) and communication overheads

Steps in Creating a Parallel Program



4 steps: **+ Scheduling**

1- Decomposition, 2- Assignment, 3- Orchestration, 4- Mapping

- **Done by programmer or system software (compiler, runtime, ...)**
- **Issues are the same, so assume programmer does it all explicitly**

Computational Problem → Parallel Algorithm → Parallel Program

Partitioning: Decomposition & Assignment

Decomposition — *Dependency Analysis/graph*

- **Break up computation into maximum number of small concurrent computations that can be combined into fewer/larger tasks in assignment step:** (i.e. parallel)
  - **Tasks may become available dynamically.**
  - **No. of available tasks may vary with time.**
  - **Together with assignment, also called *partitioning*.**

(Grain (task) size Problem (Assignment))

i.e. identify concurrency (dependency analysis) and decide level at which to exploit it.

(Task) Assignment

- **Grain-size problem:**
  - **To determine the number and size of grains or tasks in a parallel program.**
  - **Problem and machine-dependent.**
  - **Solutions involve tradeoffs between parallelism, communication and scheduling/synchronization overheads.**
- **Grain packing:**
  - **To combine multiple fine-grain nodes (parallel computations) into a coarse grain node (task) to reduce communication delays and overall scheduling overhead.**

Goal: **Enough tasks to keep processors busy, but not too many (too much overheads)**

- **No. of tasks available at a time is upper bound on achievable speedup**